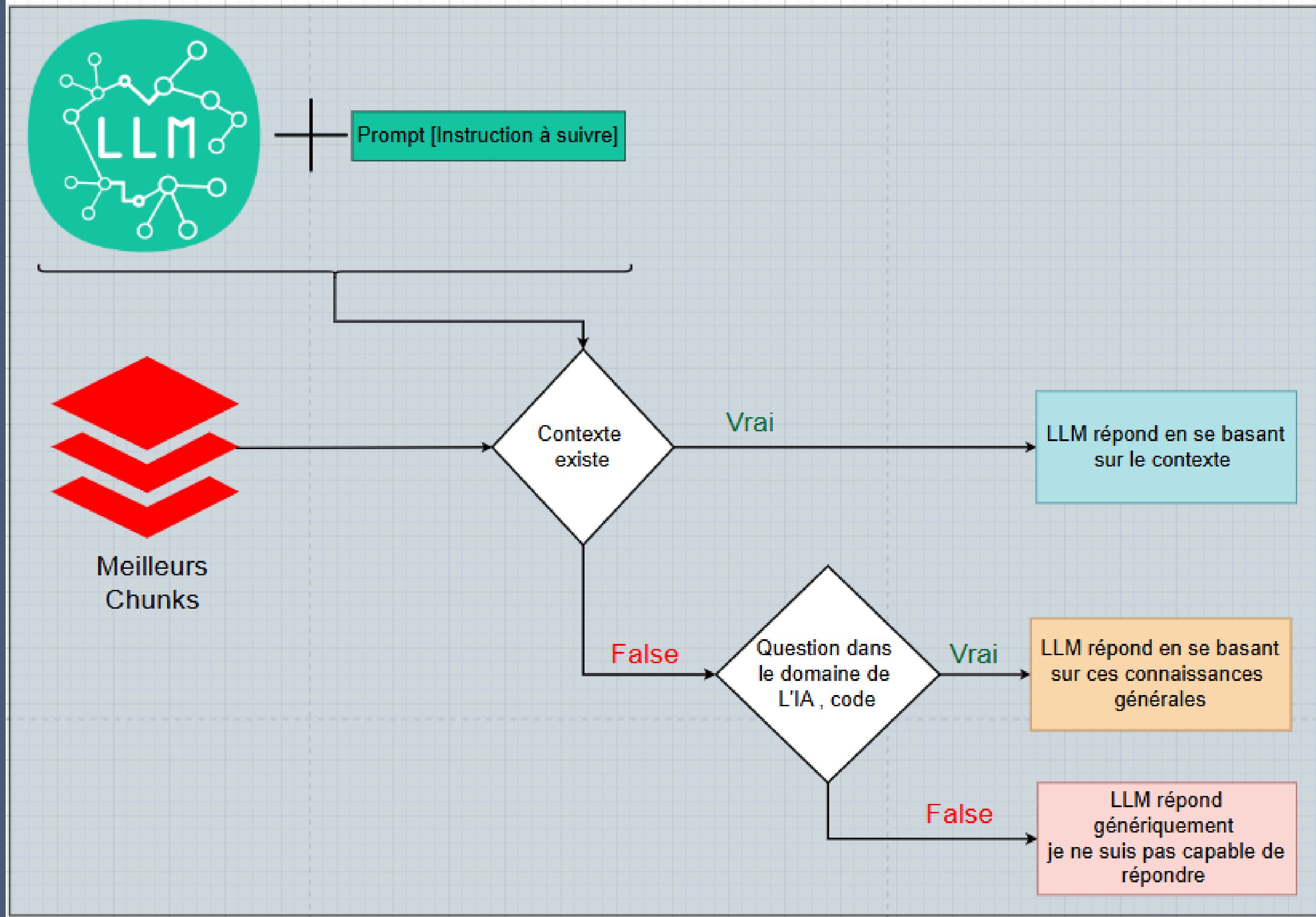
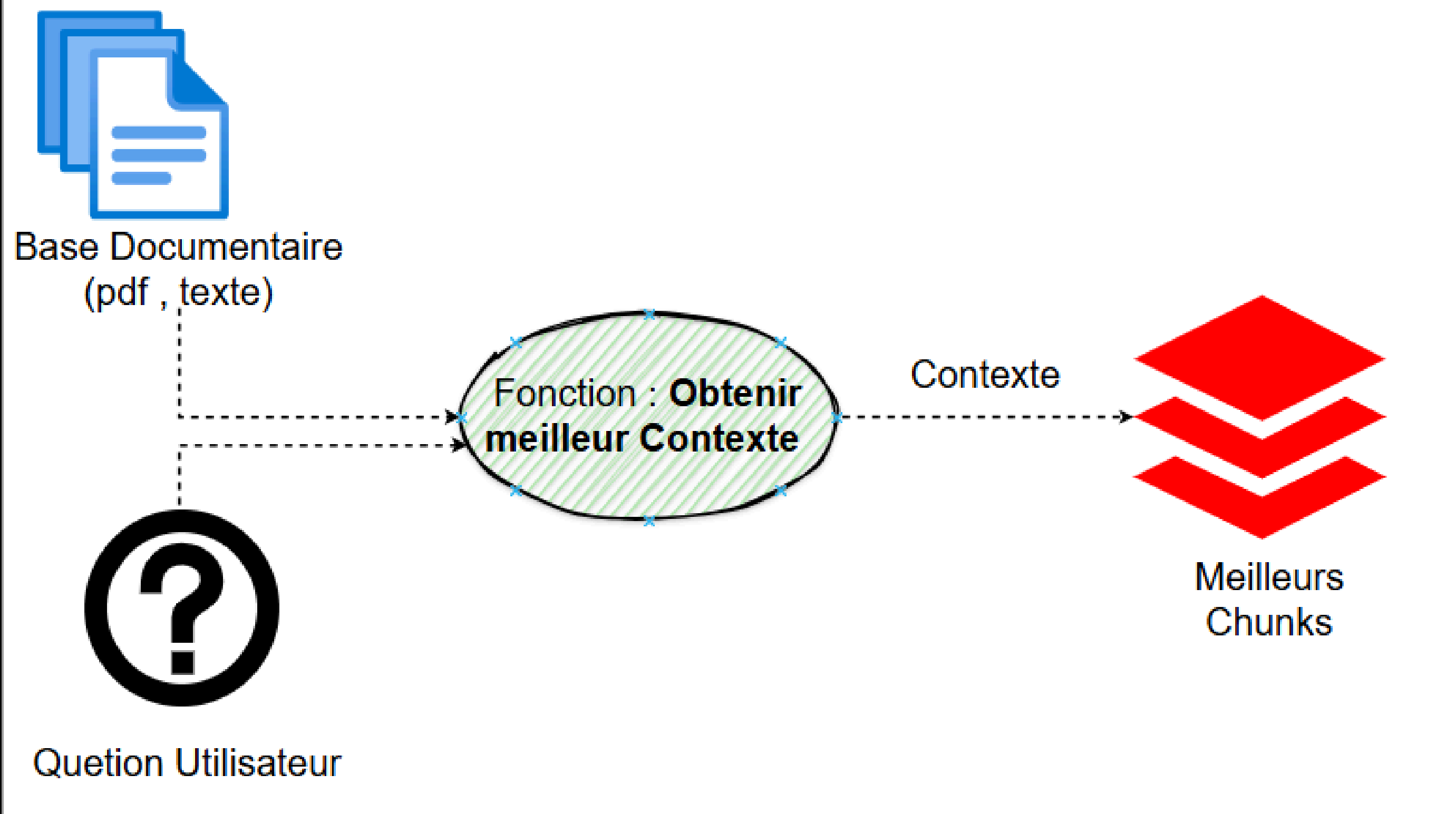


Conception du Tuteur IA

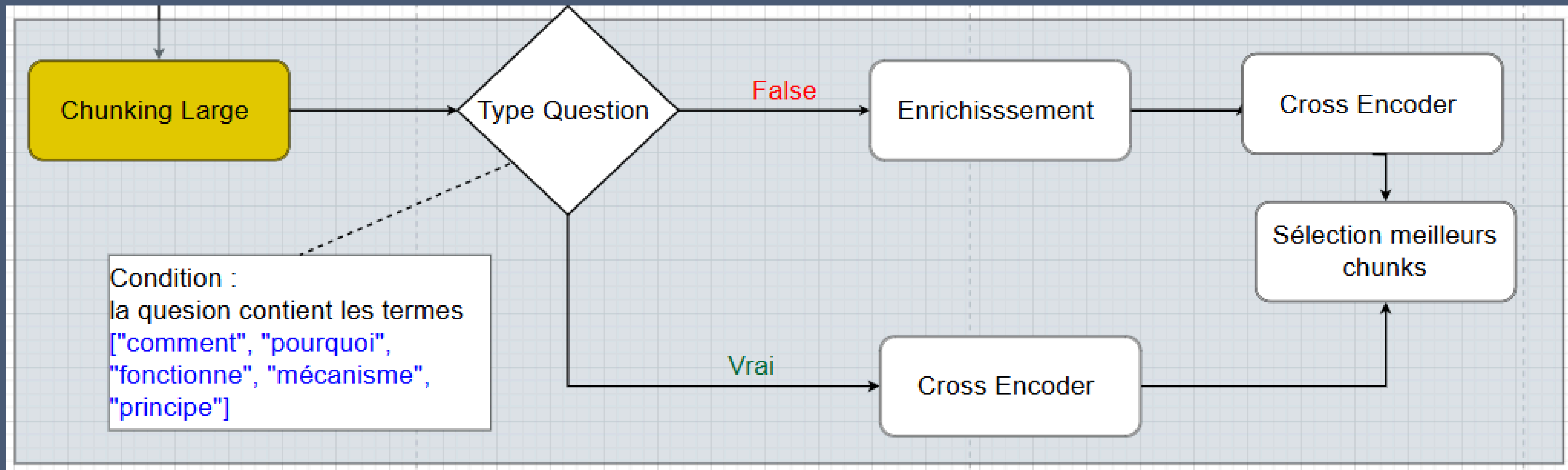
RAG pipeline



Le système commence par rechercher un contexte pertinent à partir d'une base documentaire. Si un contexte pertinent est trouvé, le LLM génère une réponse en s'appuyant sur ces informations. Dans le cas contraire, le modèle répond soit à partir de ses connaissances générales, soit indique son incapacité à répondre.



À partir de la question utilisateur et de la base documentaire (PDF, textes), le système sélectionne les chunks les plus pertinents. Ces chunks constituent le contexte qui sera ensuite fourni au LLM pour générer une réponse précise et contextualisée.



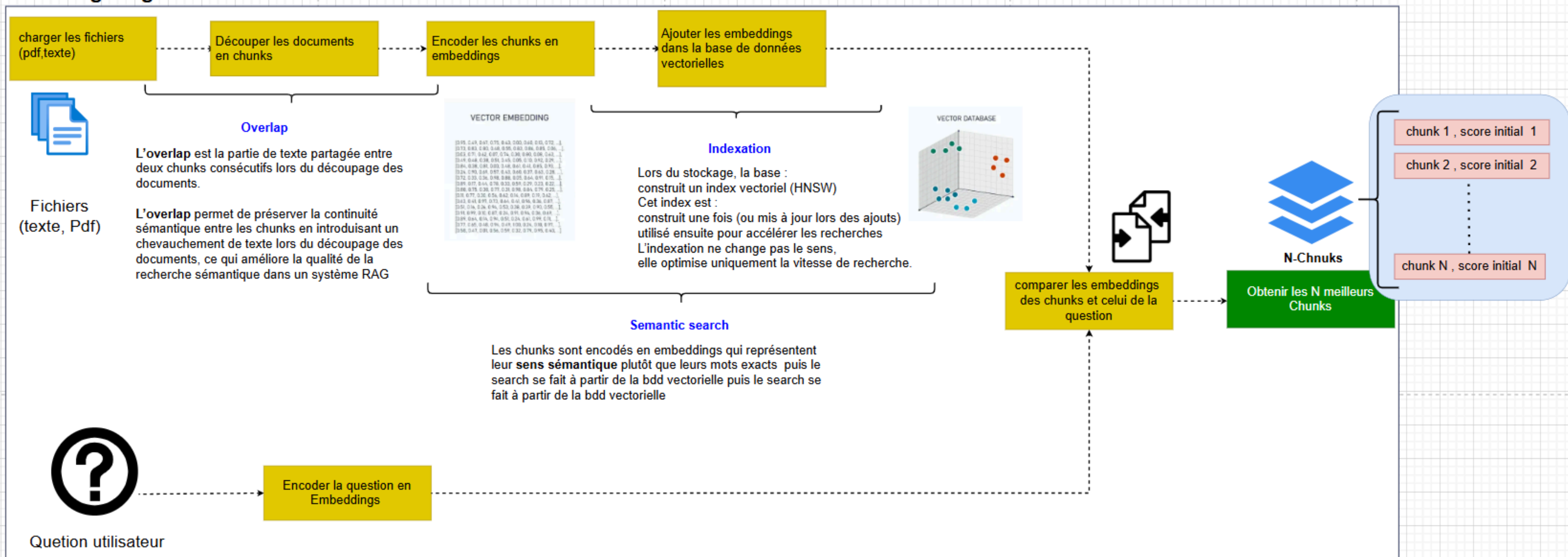
Le type de question est identifié à l'aide de règles lexicales simples (ex. comment, pourquoi, fonctionnement, mécanisme).

Ce choix permet d'adapter dynamiquement la stratégie de recherche :

- soit un reranking via un Cross-Encoder,
- soit un enrichissement de la requête + Cross-Encoder.

L'objectif est d'optimiser la pertinence du contexte tout en limitant les traitements inutiles.

Chunking Large



Les documents sont d'abord découpés en chunks avec recouvrement (overlap) afin de préserver la continuité sémantique.

Chaque chunk est ensuite encodé en embeddings et stocké dans une base de données vectorielle.

Cette indexation permet une recherche sémantique efficace basée sur le sens du texte et non uniquement sur les mots-clés

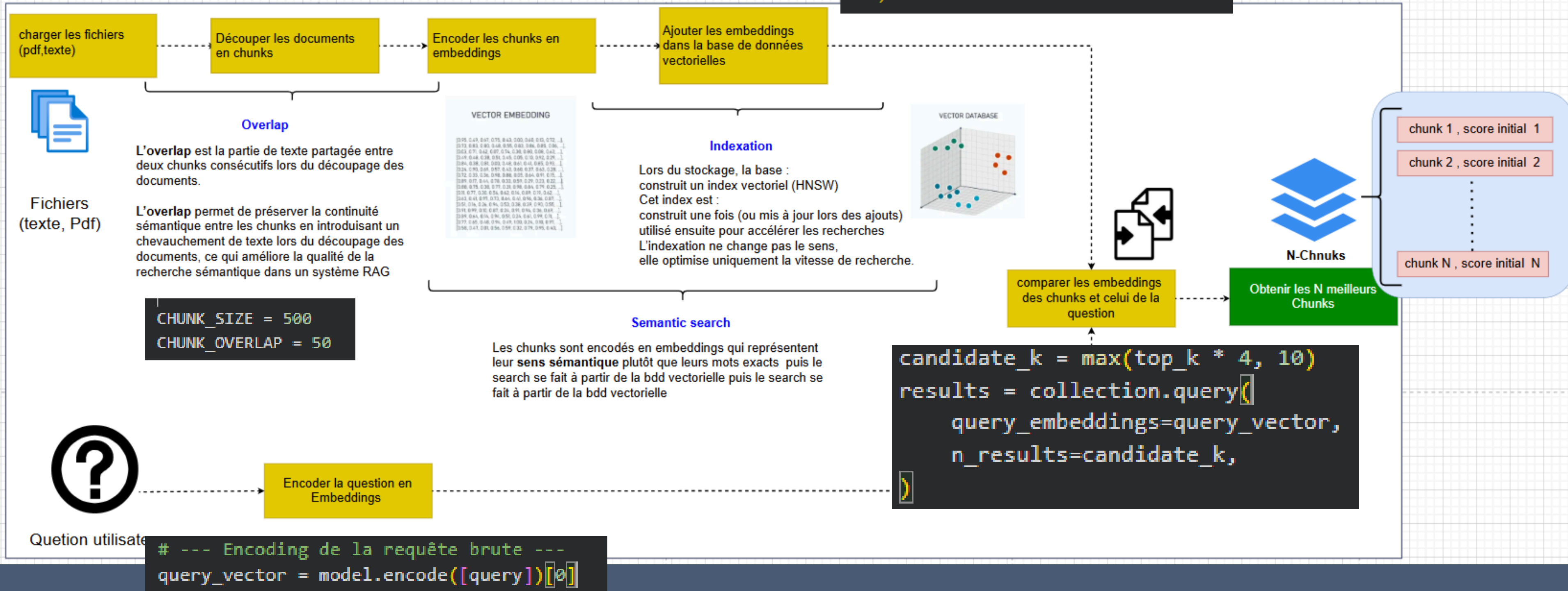
Éléments de code dans le notebook Tuteur IA

```
client = chromadb.Client()
collection = client.get_or_create_collection(
    name=COLLECTION_NAME,
    metadata={"hnsw:space": "cosine"}
)
```

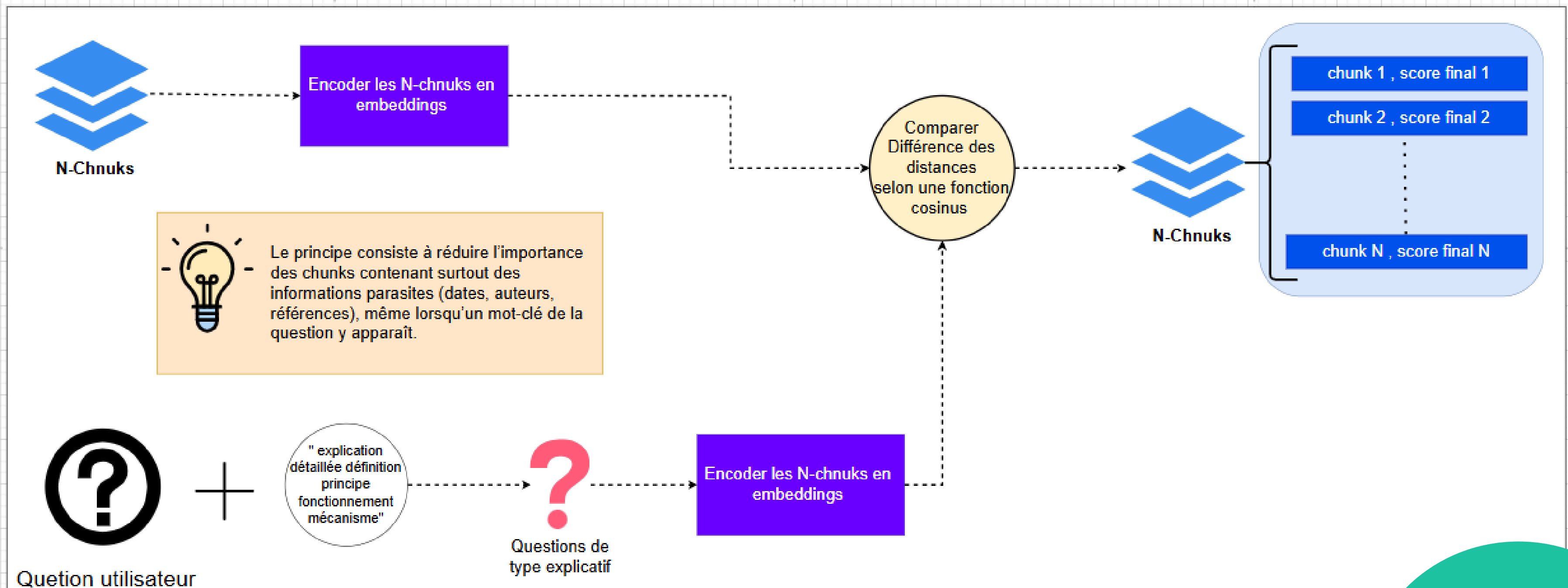
```
# Générer les embeddings
embeddings = model.encode(contents).tolist()

# Stocker dans ChromaDB
collection.add(
    ids=ids,
    embeddings=embeddings,
    documents=contents,
    metadatas=metadatas
)
```

Chunking Large



Enrichissement



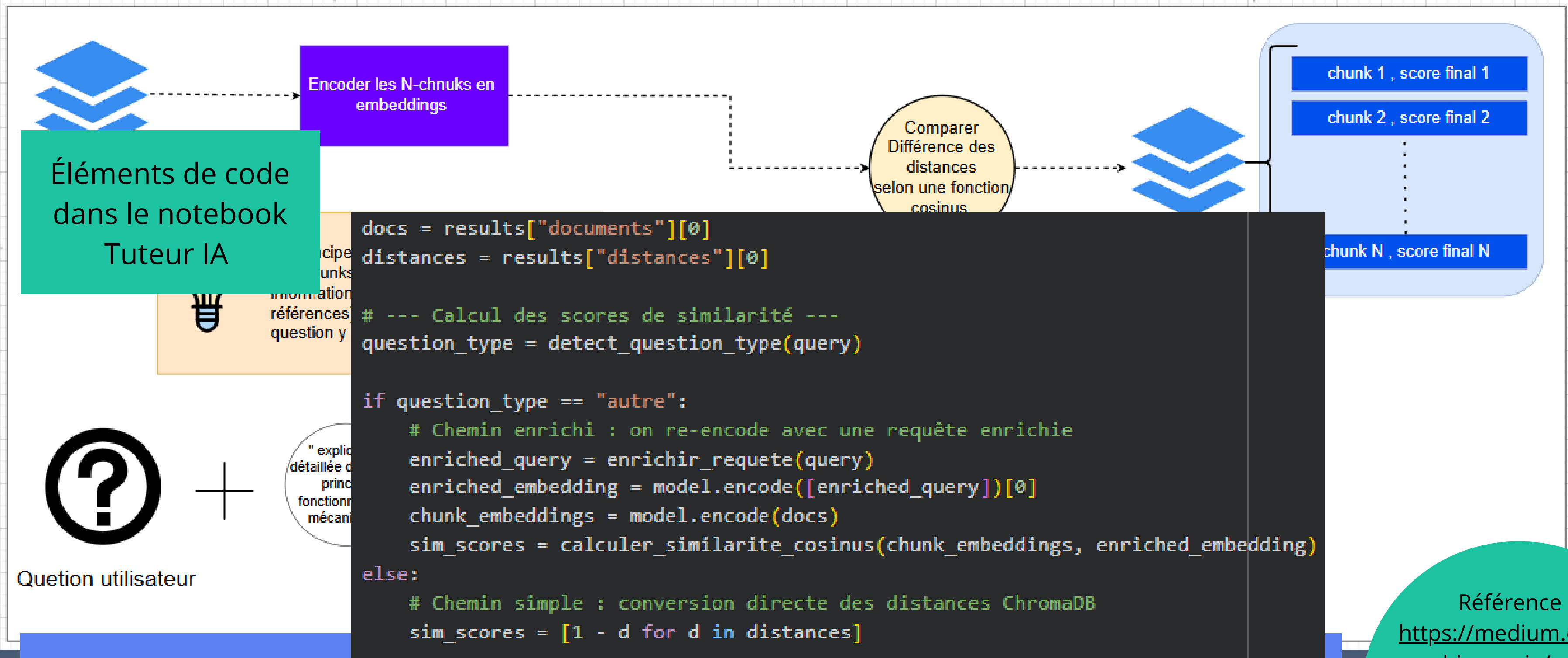
L'enrichissement de la requête consiste à reformuler ou compléter la question utilisateur avec des termes explicatifs (définition, principe, mécanisme).

Cette approche permet de réduire l'ambiguïté des requêtes courtes et d'augmenter la probabilité de récupérer des chunks réellement pertinents.

Elle est particulièrement efficace dans le cas d'un corpus restreint.

Référence :
<https://medium.com/@sahin.samia/query-expansion-in-enhancing-retrieval-augmented-generation-rag-d41153317383>

Enrichissement

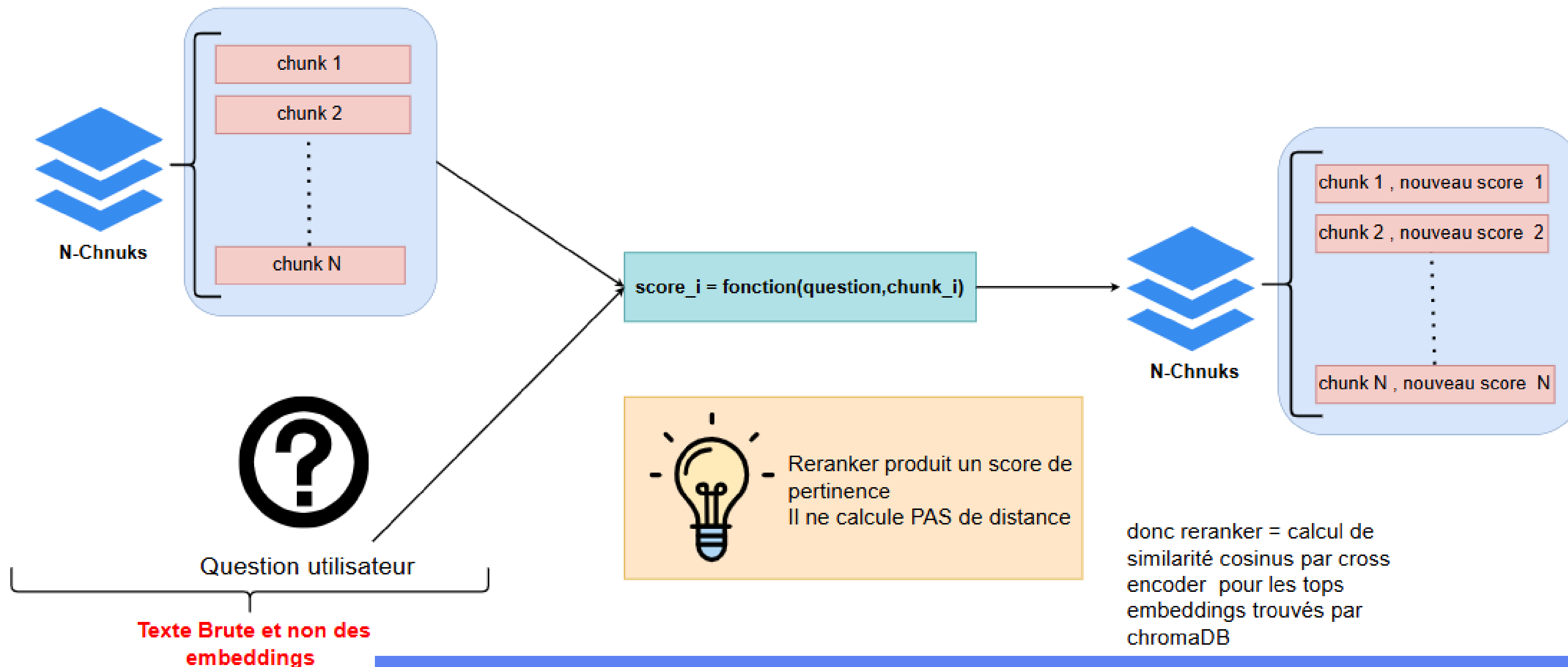


L'enrichissement de la requête consiste à reformuler ou compléter la question utilisateur avec des termes explicatifs (définition, principe, mécanisme).

Cette approche permet de réduire l'ambiguïté des requêtes courtes et d'augmenter la probabilité de récupérer des chunks réellement pertinents.

Elle est particulièrement efficace dans le cas d'un corpus restreint.

Cross Encoder



Le reranker est utilisé pour réordonner les chunks candidats en évaluant directement la pertinence entre la question et chaque chunk.

Contrairement à la recherche par embeddings, il traite les textes bruts conjointement et produit un score de pertinence.

Il ne calcule pas de distance vectorielle et sert uniquement à améliorer le classement final des chunks

Cross Encoder

Éléments de code dans le notebook Tuteur IA



Question utilis

Texte Brute et non de embeddings

```

textes = [chunk for chunk, _ in chunks_avec_scores]
paires = [(query, texte) for texte in textes]

scores_rerank = reranker.predict(paires)

indices_tries = sorted(
    range(len(textes)),
    key=lambda i: scores_rerank[i],
    reverse=True
)

return [chunks_avec_scores[i] for i in indices_tries]

```

`return [chunks_avec_scores[i] for i in indices_tries]` **ts** en évaluant directement la pertinence entre la question et chaque chunk.

Contrairement à la recherche par embeddings, il traite les textes bruts conjointement et produit un score de pertinence.

Il ne calcule pas de distance vectorielle et sert uniquement à améliorer le classement final des chunks